The Journal of Open Source Software

# BlockchainDT: A Framework for Investigating Distributed Consensus in Physical-Digital State Synchronization

**Almas Ospanov** [1¶]

**1** School of Software Engineering, Astana IT University ¶ Corresponding author

**DOI:** 10.xxxxxx/draft

**Software**

- Review
- Repository
- Archive

**Editor:**

**Submitted:** 17 December 2025
**Published:** unpublished

**License**
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

## Summary

**BlockchainDT** is an open-source research framework for investigating distributed consensus mechanisms in physical-digital state synchronization systems. The framework implements a complete digital twin architecture where multiple autonomous agents must reach consensus on the state of physical objects based on sensor measurements, predictions, and policy enforcement—all without centralized coordination.

The system combines permissioned blockchain consensus (Hyperledger Fabric), machine learning-based state prediction (XGBoost), and multi-agent coordination protocols to enable researchers to investigate fundamental questions in distributed systems: How do consensus algorithms perform when validating ML model predictions? What are the latency-accuracy trade-offs in blockchain-governed state machines? How can Byzantine fault tolerance be achieved in physical-digital synchronization scenarios?

BlockchainDT provides a concrete implementation domain—dimensional weight estimation and policy enforcement—to ground these abstract distributed systems challenges in a realistic multi-party coordination scenario. The framework includes synthetic data generation with configurable distributions, deterministic benchmarking scenarios with seeded randomness, comprehensive instrumentation for measuring consensus latency and throughput, and modular architecture enabling researchers to swap consensus protocols, prediction models, and coordination policies.

## Statement of Need

Digital twin systems that synchronize physical and digital state across distributed, mutually distrusting parties face fundamental challenges in consensus, fault tolerance, and state validation (Grieves & Vickers, 2014; Tao et al., 2018). While blockchain consensus mechanisms like PBFT (Castro & Liskov, 1999) and Raft (Ongaro & Ousterhout, 2014) are well-studied for financial transactions, their application to physical-digital state synchronization—where state updates derive from sensor measurements and ML predictions rather than simple value transfers—remains underexplored.

Existing research frameworks focus on either: (1) blockchain consensus in isolation, without physical-world grounding (Androulaki et al., 2018); (2) digital twin systems with centralized coordination (Glaessgen & Stargel, 2012); or (3) distributed machine learning without Byzantine fault tolerance (Dean et al., 2012). **No open-source framework integrates these three components** to enable investigation of consensus-driven physical-digital synchronization.

BlockchainDT addresses this gap by providing researchers with:

1. **Consensus Protocol Investigation**: Compare PBFT-style permissioned consensus with alternative approaches for validating state updates derived from ML predictions and sensor fusion

2. **Latency-Accuracy Trade-offs**: Measure how consensus overhead affects real-time state synchronization in scenarios requiring sub-second response times
3. **Byzantine Fault Tolerance in ML Validation**: Investigate how blockchain consensus can validate ML model outputs when multiple parties may submit conflicting predictions
4. **Reproducible Benchmarking**: Deterministic synthetic data generation and seeded simulation scenarios enable apples-to-apples comparison of different consensus protocols and validation strategies

## Target Research Domains

BlockchainDT enables research in:

- **Distributed Systems**: Consensus protocols for non-financial state machines, Byzantine agreement with ML-predicted state transitions
- **Digital Twin Systems**: Multi-party synchronization without centralized authority, blockchain-backed provenance for physical-digital state
- **Trustworthy ML**: Consensus-based validation of ML predictions in multi-stakeholder environments, adversarial robustness through distributed verification
- **Cyber-Physical Systems**: Real-time constraints in blockchain consensus, latency-throughput optimization for sensor-based state updates

## Related Research Software

Several tools address adjacent problems, but none combine all three core capabilities:

| Feature | SimPy | Hyperledger Caliper | Unity ML-Agents | PBFT-Go | **BlockchainDT** |
|---|---|---|---|---|---|
| Discrete-event simulation | | | | | |
| Byzantine consensus | | | | | |
| ML-based state prediction | | | | | |
| Physical-digital domain | (generic) | | | | (sensor fusion) |
| Multi-party validation | | | | | |
| Reproducible benchmarks | | | | | |

**Research Gap**: SimPy (SimPy, 2023) provides discrete-event simulation but lacks Byzantine tolerance mechanisms. Hyperledger Caliper (Hyperledger, 2023) benchmarks blockchain performance but has no physical-digital state synchronization domain. Unity ML-Agents

DRAFT

(Juliani et al., 2018) supports multi-agent RL but uses centralized coordination unsuitable for distributed consensus research. PBFT-Go (Contributors, 2023) implements Byzantine consensus but without ML-based state prediction or reproducible research testbed.

**BlockchainDT Contribution**: The first framework combining permissioned blockchain consensus, ML-driven state prediction, and reproducible multi-party benchmarking for physical-digital synchronization research.

## Software Architecture

BlockchainDT implements a modular microservices architecture enabling researchers to investigate different consensus mechanisms, prediction models, and coordination strategies:

### Consensus Layer (Hyperledger Fabric + Go)

Implements distributed ledger logic for state synchronization:

- **Immutable State Records**: All physical measurements, ML predictions, and policy decisions recorded with cryptographic timestamps
- **Smart Contract Validation**: Chaincode enforces state transition rules, validates predictions against thresholds, and manages multi-party disputes
- **Pluggable Consensus**: Framework supports PBFT-based ordering (default) with architecture enabling researchers to implement alternative protocols (Raft, PoW variants)
- **Production & Stub Modes**: Deploy to real Fabric networks or use deterministic stub for rapid prototyping

### Prediction Layer (Python + XGBoost)

ML-based state estimation with reproducible training:

- **Synthetic Data Generation**: Configurable distributions for generating realistic sensor measurements (1,000-10,000 samples)
- **Deterministic Training**: Fixed random seeds ensure reproducible model weights across experiments
- **Model Versioning**: Prediction models tagged with training hyperparameters and data distributions
- **Fast Test Mode**: Analytical fallback for unit testing without full ML inference

### Coordination Layer (Node.js + Express)

Multi-agent REST API for distributed state management:

- **State Update Endpoints**: Agents submit measurements, predictions, and policy decisions
- **Consensus Orchestration**: Coordinates blockchain transactions across multiple parties
- **Instrumentation**: Comprehensive logging of latency (consensus round-trip time), throughput (transactions/second), and accuracy (prediction error)
- **Policy Enforcement**: Pluggable rules for validating state transitions and resolving conflicts

### Analytics Layer (Python + Matplotlib)

Reproducible benchmarking and visualization:

- **Seeded Scenarios**: Three baseline scenarios (centralized, eventually-consistent, blockchain-consensus) with fixed random seeds
- **KPI Computation**: Automated calculation of latency, throughput, prediction MAE, dispute rates, and cost metrics
- **Publication Plots**: Publication-ready visualizations comparing consensus protocols

DRAFT

# Research Applications

### Consensus Protocol Comparison

Researchers can compare latency-throughput trade-offs across three baseline scenarios:

```
# Run three consensus scenarios (1000 iterations each, deterministic seed)
cd erp-prototype && python demo/demo_scenarios.py --seed 42 --limit 1000

# Generate comparative plots
python demo/plot_kpis.py
```

**Current v1.0.0 scenarios**: - `baseline_a`: Eventually-consistent (no blockchain) - baseline for comparison - `baseline_b`: Centralized coordination - single authority model - `proposed`: PBFT-style consensus - distributed Byzantine tolerance

**Research questions enabled**: - How does consensus overhead affect latency and throughput? - What is the accuracy vs. cost trade-off for different coordination strategies? - Can blockchain consensus meet real-time digital twin requirements?

**Expected outputs**: `demo/results_kpi.csv` (3000 rows), `demo/summary_results.csv` (aggregated metrics), publication-ready plots showing latency distributions and throughput comparisons.

# Testing and Quality Assurance

- **Comprehensive Test Suite**: 37 Jest tests covering API endpoints, consensus integration, ML prediction, and error paths (100% statement coverage, 85.7% branch coverage)
- **Deterministic Scenarios**: All benchmarks use fixed random seeds (default: seed=42) for reproducibility
- **CI/CD Pipeline**: GitHub Actions runs full test suite + scenario benchmarks on every commit
- **Synthetic Data Validation**: Generated datasets validated against statistical properties (distribution moments, outlier rates)

# Implementation Notes

### Reproducibility Guarantees

All components designed for deterministic execution:

- Random seeds fixed in data generation, ML training, and scenario simulation
- Model hyperparameters version-controlled in `optimization/dw_model.json`
- Blockchain stub mode provides deterministic transaction ordering
- Docker containers pin dependency versions

### Extensibility for Researchers

Modular design enables swapping components:

- **Consensus Protocol**: Replace Fabric with Raft, Tendermint, or custom protocols via adapter pattern
- **Prediction Model**: Swap XGBoost with neural networks, GPs, or ensemble methods
- **Coordination Policy**: Implement custom state validation rules in chaincode
- **Benchmarking Scenarios**: Add new scenarios in `demo/` with different data distributions, attack models, or network topologies

### Performance Characteristics

Measured on 4-core Intel i7, 16GB RAM, Ubuntu 22.04:

DRAFT

- **Consensus Latency**: 1.47s mean (blockchain mode), 0.52s (stub mode)
- **Throughput**: 58 transactions/second (blockchain), 120 tx/s (stub)
- **Prediction Latency**: 12ms per item (XGBoost), 3ms (fast mode)
- **Memory Footprint**: 450MB (full stack), 120MB (stub only)


## Acknowledgements

We acknowledge the Hyperledger Fabric project, the XGBoost development community, the JOSS reviewers, and the open-source distributed systems research community. This research was conducted at the School of Software Engineering, Astana IT University.


## References


Androulaki, E., Barger, A., Bortnikov, V., Cachin, C., Christidis, K., Caro, A. D., Enyeart, D., Ferris, C., Laventman, G., Manevich, Y., Muralidharan, S., Murthy, C., Nguyen, B., Sethi, M., Singh, G., Smith, K., Sorniotti, A., Stathakopoulou, C., Vukolic, M., … Yellick, J. (2018). Hyperledger fabric: A distributed operating system for permissioned blockchains. *Proceedings of the Thirteenth EuroSys Conference (EuroSys)*, 30:1–30:15. https://doi.org/10.1145/3190508.3190538

Castro, M., & Liskov, B. (1999). Practical byzantine fault tolerance. *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*, 173–186. https://pmg.csail.mit.edu/papers/osdi99.pdf

Contributors, P.-G. (2023). *PBFT-go: Go implementation of practical byzantine fault tolerance*. Software. https://github.com/gitferry/pbft-go

Dean, J., Corrado, G. S., Monga, R., Chen, K., Devin, M., Le, Q. V., Mao, M. Z., Ranzato, M., Senior, A., Tucker, P., Yang, K., & Ng, A. Y. (2012). Large scale distributed deep networks. *Advances in Neural Information Processing Systems (NeurIPS)*, *25*, 1223–1231. https://papers.nips.cc/paper/4687-large-scale-distributed-deep-networks.pdf

Glaessgen, E., & Stargel, D. (2012). The digital twin paradigm for future NASA and u.s. Air force vehicles. *53rd AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics and Materials Conference*, 1818. https://doi.org/10.2514/6.2012-1818

Grieves, M., & Vickers, J. (2014). Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems. *Transdisciplinary Perspectives on Complex Systems*, 85–113. https://doi.org/10.1007/978-3-319-38756-7_4

Hyperledger. (2023). *Hyperledger caliper: Blockchain performance benchmark framework*. Software. https://hyperledger.github.io/caliper/

Juliani, A., Berges, V.-P., Teng, E., Cohen, A., Harper, J., Elion, C., Goy, C., Gao, Y., Henry, H., Mattar, M., & Lange, D. (2018). Unity: A general platform for intelligent agents. *arXiv Preprint arXiv:1809.02627*. https://arxiv.org/abs/1809.02627

Ongaro, D., & Ousterhout, J. (2014). In search of an understandable consensus algorithm. *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC)*, 305–320. https://raft.github.io/raft.pdf

SimPy, T. (2023). *SimPy: Discrete-event simulation for python*. Software. https://simpy.readthedocs.io/

Tao, F., Cheng, J., Qi, Q., Zhang, M., Zhang, H., & Sui, F. (2018). Digital twin in industry: State-of-the-art. *IEEE Transactions on Industrial Informatics*, *15*(4), 2405–2415. https://doi.org/10.1109/TII.2018.2873186


DRAFT